OFF-BY-ONE 2026 - SINGAPORE

From 1-day vulnerability analysis to Copy Fail

Combining AI with Human Insight

Taeyang Lee

©2026. Theori. All rights reserved.

theori

Who Am I

Security researcher at Theori.



Taeyang Lee / 이태양

- Senior Researcher, Theori
- 0wn@theori.io
- X / Discord: @5unkn0wn

Recent & Interested work

- Web2, kernel and low-level security at Theori
- CVE-2026-31431 (Copy Fail)
- AI-assisted vulnerability research
- Ex Heavy CTF player (The Duck and MMM)

©2026. Theori. All rights reserved.

Agenda

1. **Background: the crypto subsystem**
   The API, its callers and the way it handles data
2. **Analyzing a 1-day: CVE-2025-40019**
   An unsigned underflow in a scatterlist copy
3. **Tracing a variant: CVE-2026-23060**
   Searching for similar bugs with AI
4. **Discovering Copy Fail**
   A write into a file page supplied through splice()
5. **AI with human expertise**
   How I chose hypotheses and reviewed the results

The 1-day analysis led to a variant search, followed by the investigation that found Copy Fail.

COPY FAIL

Root through a page-cache overwrite

During AF_ALG decryption, authencesn writes 4 bytes immediately after the caller’s output. splice() can place a file’s page-cache page at that position, even when the caller has only read access to the file. CVE-2026-31431.

- Combines an authencesn scratch write with a 2017 change in algif_aead
- A 732-byte Python exploit worked on Ubuntu, Amazon Linux, RHEL and SUSE
- The write remains in the page cache until eviction, without changing the file on disk

1

Background: the crypto subsystem

IPsec, dm-crypt and TLS use the kernel crypto API. AF_ALG also exposes it to unprivileged processes.

- The services the crypto API provides
- Using AF_ALG from userspace
- Data handling with scatterlists

The kernel crypto API

Kernel subsystems request cryptographic algorithms by name. The crypto API selects an implementation.

01

ALGORITHMS

Crypto primitives

skcipher · aead · shash
ahash · rng · akcipher · kpp

02

CALLERS

Kernel callers

IPsec (ESP) · dm-crypt
fscrypt · kTLS · AF_ALG

03

IMPLEMENTATIONS

Pluggable backends

generic C · arch code (AES-NI)
hardware offload · sync or async

AF_ALG provides userspace access to these algorithms.

©2026. Theori. All rights reserved.

Algorithm composition

crypto_template creates an algorithm instance from child algorithms named by the caller.

Template examples

```
cbc(aes)
  |   |
  |   +------ cipher   : AES block cipher
  +---------- skcipher : CBC mode

essiv( cbc(aes), sha256 )
  |      |   |     |
  |      |   |     +-- shash   : IV salt = SHA256(key)
  |      |   +-------- cipher  : AES block cipher
  |      +------------ skcipher: CBC mode
  +------------------- template: ESSIV IV generation

authencesn( hmac(sha256), cbc(aes) )
  |            |             |
  |            |             +-- encryption
  |            +---------------- authentication
  +----------------------------- ESP/ESN AAD handling
```

crypto_template

- Builds an instance from the names of its child algorithms

crypto_instance / crypto_spawn

- The resulting algorithm and its typed references to the children

Template assumptions

- The algorithm name determines the template composition
- Some templates assume the data layout used by their original caller

©2026. Theori. All rights reserved.

The AF_ALG interface

A userspace program selects an algorithm, supplies a key and sends data through an AF_ALG socket.

AF_ALG example

```
int tfm = socket(AF_ALG, SOCK_SEQPACKET, 0);

struct sockaddr_alg sa = {
    .salg_family = AF_ALG,
    .salg_type   = "aead",
    .salg_name   = "gcm(aes)",
};
bind(tfm, (struct sockaddr *)&sa, sizeof(sa));
setsockopt(tfm, SOL_ALG, ALG_SET_KEY, key, keylen);

int op = accept(tfm, NULL, 0);

sendmsg(op, &msg, 0);    /* data + ALG_SET_OP, IV, ASSOCLEN */
read(op, out, outlen);   /* the transformed bytes come back */
```

What the caller controls

algorithm name	any template composition
key material	every component key
operation	encrypt or decrypt
assoclen	**set per request, by a cmsg**
cryptlen	**set by how much data you send**
IV, authsize	within the algorithm's limits
input data	sent with sendmsg()

CONFIG_CRYPTO_USER_API_AEAD is on in most distro kernels. No capability and no namespace is required to open the socket.

©2026. Theori. All rights reserved.

An in-kernel AES-CBC request

This example passes input and output buffers to the crypto API as scatterlists.

cbc(aes) · in-kernel skcipher

```
struct crypto_skcipher  *tfm;
struct skcipher_request *req;
struct scatterlist       sg_in, sg_out;

tfm = crypto_alloc_skcipher("cbc(aes)", 0, 0);
crypto_skcipher_setkey(tfm, key, 16);
req = skcipher_request_alloc(tfm, GFP_KERNEL);

sg_init_one(&sg_in,  plaintext,  len);
sg_init_one(&sg_out, ciphertext, len);

skcipher_request_set_crypt(req, &sg_in, &sg_out,
                           len, iv);
crypto_skcipher_encrypt(req);  /* callback + wait elided */
```

Request fields

"cbc(aes)"	resolved by name at alloc time
setkey	16 bytes, once per transform
sg_init_one	one buffer becomes a one-entry list
sg_in, sg_out	**scatterlists for input and output**
len	**a byte count spanning the whole list**
iv	16 bytes, supplied per request
each entry	**names a page, an offset and a length**

sg_init_one() wraps a buffer in one entry. Other callers use several entries to describe data across pages.

©2026. Theori. All rights reserved.

struct scatterlist

The crypto API uses scatterlists to describe buffers. Each data entry identifies a page, an offset and a length.

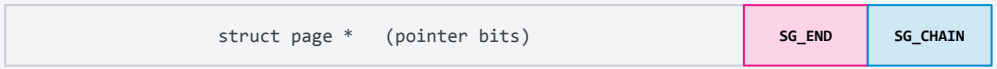
include/linux/scatterlist.h

```
struct scatterlist {
    unsigned long   page_link;   /* struct page * | flag bits    */
    unsigned int    offset;      /* byte offset inside the page  */
    unsigned int    length;      /* bytes this entry contributes */
};

#define SG_CHAIN  0x01UL   /* page_link points at another array */
#define SG_END    0x02UL   /* last entry of the list            */

struct page        *sg_page(struct scatterlist *sg);
struct scatterlist *sg_next(struct scatterlist *sg);
```

page_link



a struct page * is at least 8-byte aligned, so the two low bits are free

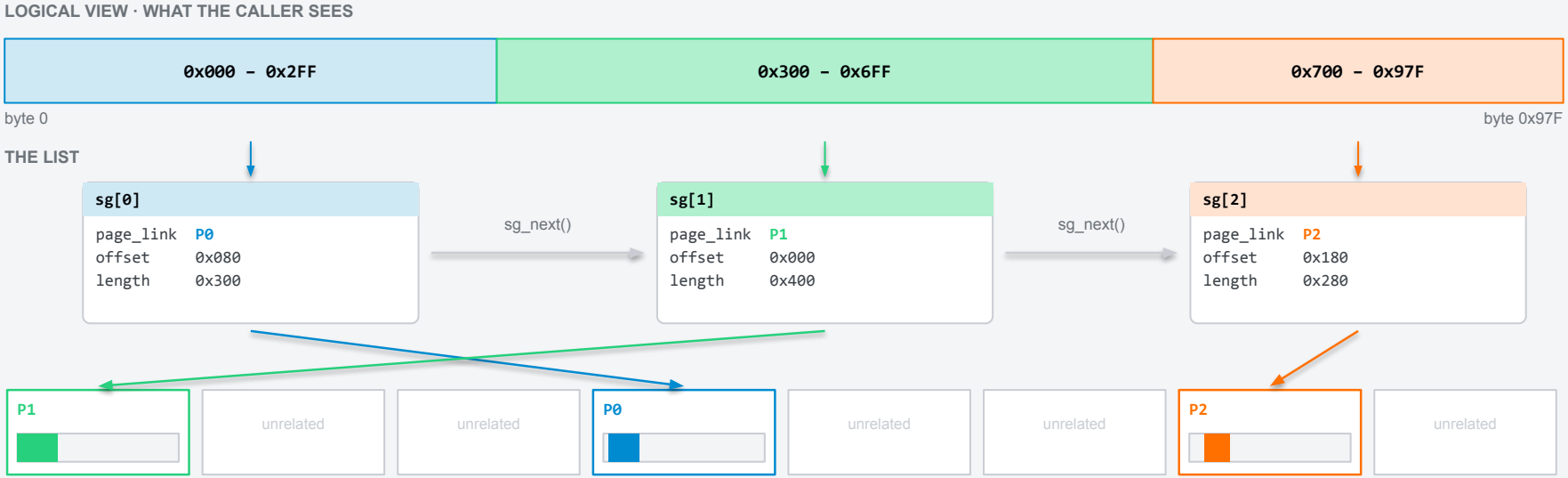
What one entry names

`page_link`	a struct page *, with two low flag bits
`offset`	where the fragment starts inside page
`length`	how many bytes this entry contributes
`sg_next()`	the next entry, a chained array, or NULL

©2026. Theori. All rights reserved.

Mapping a byte range to pages

The caller counts bytes from zero. scatterwalk turns that count into a page and an offset inside it.



Logical bytes 0x2FF and 0x300 are adjacent. Physically they are P0 + 0x37F and P1 + 0x000.

©2026. Theori. All rights reserved.

Walking a scatterlist with scatterwalk_ffwd()

scatterwalk_ffwd() reaches a starting offset by subtracting one entry at a time.

crypto/scatterwalk.c

```
struct scatterlist *scatterwalk_ffwd(struct scatterlist dst[2],
                                     struct scatterlist *src,
                                     unsigned int len)
{
    for (;;) {
        if (!len)
            return src;
        if (src->length > len)
            break;
        len -= src->length;    /* skip this entry entirely */
        src = sg_next(src);    /* and move to the next one */
    }
    ...
}
```

Walking to offset 0x500

1 `len = 0x500` — `sg[0] length 0x300`
`0x300 ≤ 0x500` subtract, then sg_next()

2 `len = 0x200` — `sg[1] length 0x400`
`0x400 > 0x200` stop inside this entry

`start = sg[1] + 0x200`

checked each entry's own length
not checked whether len fits the list at all
not checked whether sg_next() returned NULL

The helper checks each entry's length. It assumes the caller has already checked that the requested offset fits the list.

©2026. Theori. All rights reserved.

2

Analyzing a 1-day: CVE-2025-40019

Muhammad Alifa Ramdhan (STAR Labs SG) reported and exploited this vulnerability. The upstream fix landed in October 2025.

- The patch and the unsigned subtraction it checks
- ESSIV's AAD layout and the input AF_ALG accepts
- Parts of the exploit I reused in the later research

The CVE-2025-40019 patch

crypto: essiv - Check ssize for decryption and in-place encryption. Reported by Muhammad Alifa Ramdhan (STAR Labs SG)

```
diff --git a/crypto/essiv.c b/crypto/essiv.c
index d003b78fcd855..a47a3eab69351 100644
--- a/crypto/essiv.c
+++ b/crypto/essiv.c
@@ -186,9 +186,14 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
        const struct essiv_tfm_ctx *tctx = crypto_aead_ctx(tfm);
        struct essiv_aead_request_ctx *rctx = aead_request_ctx(req);
        struct aead_request *subreq = &rctx->aead_req;
+       int ivsize = crypto_aead_ivsize(tfm);
+       int ssize = req->assoclen - ivsize;
        struct scatterlist *src = req->src;
        int err;

+       if (ssize < 0)
+               return -EINVAL;
+
        crypto_cipher_encrypt_one(tctx->essiv_cipher, req->iv, req->iv);

        /*
@@ -198,19 +203,12 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
         */
        rctx->assoc = NULL;
        if (req->src == req->dst || !enc) {
-               scatterwalk_map_and_copy(req->iv, req->dst,
-                                        req->assoclen - crypto_aead_ivsize(tfm),
-                                        crypto_aead_ivsize(tfm), 1);
+               scatterwalk_map_and_copy(req->iv, req->dst, ssize, ivsize, 1);
        } else {
                u8 *iv = (u8 *)aead_request_ctx(req) + tctx->ivoffset;
-               int ivsize = crypto_aead_ivsize(tfm);
-               int ssize = req->assoclen - ivsize;
                struct scatterlist *sg;
                int nents;

-               if (ssize < 0)
-                       return -EINVAL;
-
```

What the patch does

- The code passed `assoclen - ivsize` to `scatterwalk_map_and_copy()` without a check
- When `assoclen` is smaller than `ivsize`, the unsigned subtraction underflows
- The existing check covered out-of-place encryption only

Moving the check to the start also covers decryption and in-place encryption.

©2026. Theori. All rights reserved.

assoclen − ivsize wraps to 0xfffffff0

Both operands are unsigned, and the result becomes the offset the copy starts from.

req->assoclen	−	ivsize	=	unsigned result
0x00000000		0x00000010		0xfffffff0

crypto/essiv.c : essiv_aead_crypt()

```
scatterwalk_map_and_copy(req->iv,     /* 16 encrypted IV bytes */
                         req->dst,    /* the destination list  */
                         0xfffffff0,  /* start                 */
                         16, 1);      /* nbytes, out           */
```

Where the two values come from

assoclen	set per request by ALG_SET_AEAD_ASSOCLEN, including 0
ivsize	fixed by the selected algorithm, with 16 bytes for cbc(aes)
!enc	ALG_SET_OP = ALG_OP_DECRYPT selects the branch with no check

The operation copies 16 bytes starting at offset 0xfffffff0.

©2026. Theori. All rights reserved.

The walk runs past the end of the list

scatterwalk_ffwd() subtracts one entry at a time and never compares the offset against the whole list.

crypto/scatterwalk.c : scatterwalk_ffwd()

```
for (;;) {
    if (!len)               /* 0xffffffff0 never reaches 0 */
        return src;
    if (src->length > len)  /* 0x100 > 0xffffffff0 is false */
        break;
    len -= src->length;
    src = sg_next(src);     /* walks past the last entry   */
}
```

WHAT THE POC SENDS

name	essiv(authenc(hmac(sha256),cbc(aes)),sha256)
assoclen	0
op	ALG_OP_DECRYPT
iovec	one 0x100-byte segment

Kernel crash

Linux 6.12.48

```
BUG: kernel NULL pointer dereference, address: 000000000000000c
RIP: 0010:scatterwalk_ffwd+0x30/0xc0
RBP: 00000000fffffef0
Call Trace:
 scatterwalk_map_and_copy+0x56/0xa0
 essiv_aead_crypt+0x1aa/0x310
 aead_recvmsg+0x59b/0x670
 __x64_sys_recvfrom+0x24/0x30
```

- sg_next() returned NULL, and the loop read length at offset 0xc
- RBP holds 0xfffffef0 after subtracting the single 0x100-byte entry

©2026. Theori. All rights reserved.

Exploit strategy

The exploit uses an uninitialized destination list and heap grooming to direct the 16-byte copy into a page table.

EXPLOIT STEPS

01 UNINITIALISED DESTINATION

A request with an uninitialized list

WHY

The large offset runs past a normal list. An uninitialized list lets the exploit reuse entries left by earlier allocations.

a tag-only decrypt: outlen = 0, so the array stays residue

02 SLAB GROOMING

Arrange the entries left in the allocation.

WHY

The inherited list must contain entries and chain links that direct the walk to the target page.

rsgl allocations placed and freed, then one reclaimed

03 PAGE-TABLE WRITE

Reclaim the target page as a page table.

WHY

The exploit releases pipe pages and reclaims them for page tables while the inherited list still references them.

a 16-byte write across two page-table entries

The exploit first prepares the fake destination list, then reclaims its target page as a page table.

©2026. Theori. All rights reserved.

STEP 1 OF 3 · UNINITIALISED dst

An uninitialized destination scatterlist

A tag-only decrypt produces no output. AF_ALG keeps the tag in the TX list but skips initialising the RX entries that req->dst points to.

crypto/algif_aead.c : _aead_recvmsg()

```
static int _aead_recvmsg(struct socket *sock, struct msghdr *msg,
                         size_t ignored, int flags)
{
        unsigned int as = crypto_aead_authsize(tfm);   /* 0x20 */

        used = ctx->used;                    /* 0x20: one tag, nothing else */

        outlen = used - as;                  /* 0x20 - 0x20 = 0, means there is no output data */

        /* Allocate cipher request for current operation. */
        areq = af_alg_alloc_areq(sk, sizeof(struct af_alg_async_req) +
                                 crypto_aead_reqsize(tfm));

        /* outlen == 0: succeeds without initializing RX entries */
        err = af_alg_get_rsgl(sk, msg, flags, areq, outlen, &usedpages);
        if (err)
                goto free;
}
```

- **`maxsize`** is caller's **`outlen`**, and **`len`** starts at **`0`**.
- When **`outlen`** is **`0`**, the loop is skipped and the scatterlist remains uninitialized.

1. Request allocation without zeroing

```
struct af_alg_async_req *af_alg_alloc_areq(struct sock *sk, unsigned int areqlen)
{
        areq = sock_kmalloc(sk, areqlen, GFP_KERNEL);   /* no __GFP_ZERO */
        areq->areqlen = areqlen;
        areq->sk = sk;
        areq->first_rsgl.sgl.sgt.sgl = areq->first_rsgl.sgl.sgl;
}
```

2. Destination-list initialisation

```
int af_alg_get_rsgl(struct sock *sk, struct msghdr *msg, int flags,
                    struct af_alg_async_req *areq, size_t maxsize, size_t *outlen)
{
        size_t len = 0;
        while (maxsize > len && msg_data_left(msg)) {
                sg_init_table(rsgl->sgl.sgt.sgl, ALG_MAX_PAGES);
                err = extract_iter_to_sg(&msg->msg_iter, seglen, ...);
        }
        *outlen = len;      /* zero for this request */
        return 0;
}
```

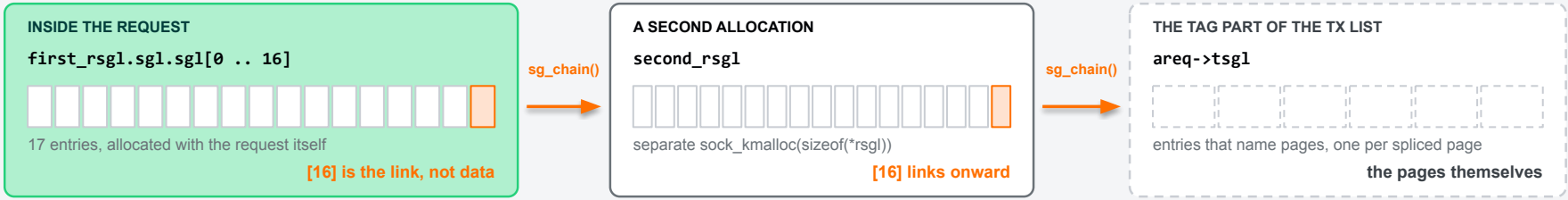
©2026. Theori. All rights reserved.

STEP 1 OF 3 · UNINITIALISED dst

first_rsgl, second_rsgl and tsgl

first_rsgl is an array inside the request. Anything past 16 entries needs a second rsgl at its own address, and the 17th entry is the link to it.

HOW THE DESTINATION LIST IS PUT TOGETHER



include/crypto/if_alg.h

```
#define ALG_MAX_PAGES   16
struct af_alg_sgl {
        struct sg_table sgt;
        struct scatterlist sgl[ALG_MAX_PAGES + 1];   /* 17 entries */
};
struct af_alg_async_req {
        struct af_alg_rsgl first_rsgl;   /* First RX SG - this is req->dst */
        struct scatterlist *tsgl;        /* per request TX SGL */
};
```

Separate allocations

- the request, second RX list and TX list use separate allocations
- each allocation can be freed and reused independently
- sg_next() follows the stored chain links between them

©2026. Theori. All rights reserved.

AF_ALG accepts splice()

splice_to_socket() sends MSG_SPLICE_PAGES through sendmsg(). AF_ALG stores references to the supplied pages.

1. HOW splice() GETS THERE

fs/splice.c : splice_to_socket()

```
msg.msg_flags = MSG_SPLICE_PAGES;
        ...
iov_iter_bvec(&msg.msg_iter, ITER_SOURCE, bvec, bc,
              len - remain);
ret = sock_sendmsg(sock, &msg);
```

2. WHAT AF_ALG DOES WITH IT

crypto/af_alg.c : af_alg_sendmsg()

```
if (msg->msg_flags & MSG_SPLICE_PAGES) {
        ...
        plen = extract_iter_to_sg(&msg->msg_iter, len, &sgtable,
                                  MAX_SGL_ENTS - sgl->cur, 0);
        for (; sgl->cur < sgtable.nents; sgl->cur++)
                get_page(sg_page(&sg[sgl->cur]));
}
```

Page references in the TX list

- the TX scatterlist holds references to the supplied pages
- get_page() gives the socket its own reference
- splice() lets the caller supply pages to the crypto request

The request retains references to the pages supplied through splice().

©2026. Theori. All rights reserved.

Page references through splice()

splice() passes page references from a file through a pipe to the AF_ALG socket.

WHAT USER SPACE ASKS FOR

File-to-pipe and pipe-to-socket calls

```
int p[2];
pipe(p);
/* no read(), no write(): only page references move */
splice(fd,   NULL, p[1], NULL, 0x1000, 0);   /* file -> pipe   */
splice(p[0], NULL, sock, NULL, 0x1000, 0);   /* pipe -> socket */
```

WHAT THE PIPE HOLDS

include/linux/pipe_fs_i.h

```
struct pipe_buffer {
        struct page *page;      /* the page itself, by reference */
        unsigned int offset;    /* where the data starts in it   */
        unsigned int len;       /* how many bytes it contributes */
        ...
};
```

Page types

- the pipe stores page references without copying the data through userspace
- these can refer to anonymous pipe pages or a file's page cache
- the receiving socket takes references to the supplied pages

This exploit uses anonymous pipe pages that can later be freed and reclaimed.

©2026. Theori. All rights reserved.

The required heap layout

Freeing the prepared request releases its page references. Old page addresses remain in freed SGL memory while the anonymous pages can be reused.

SCATTERLIST WALK



REQUIRED CONDITIONS

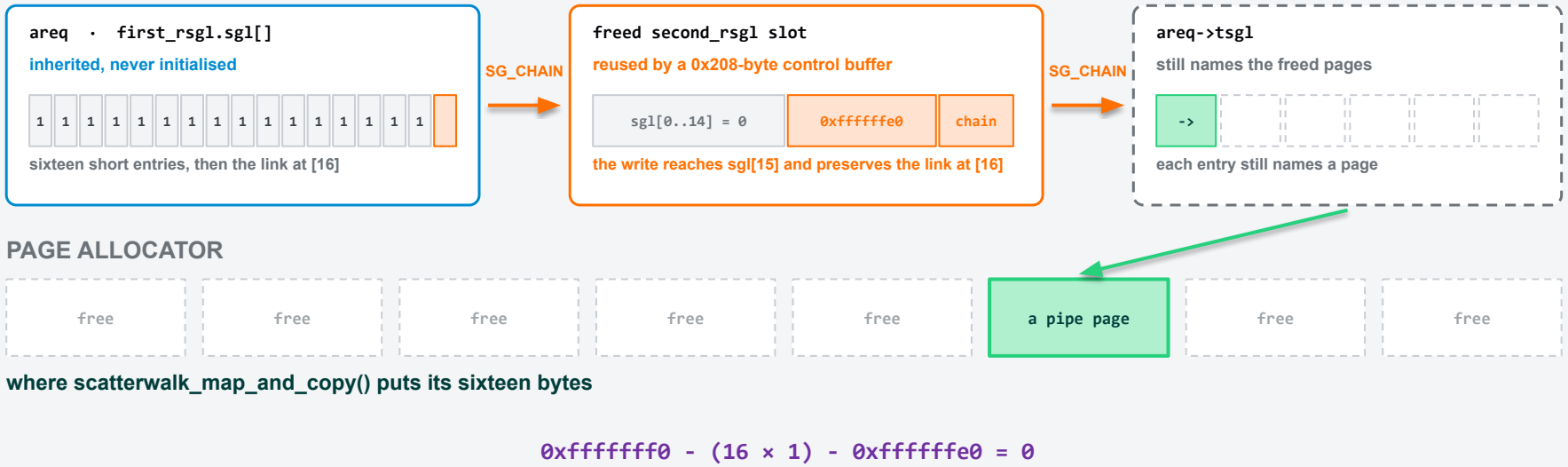
- the request reuses an allocation whose contents the exploit prepared
- the 0x208-byte socket buffer reaches the forged length while preserving the next chain link
- the pipe pages are released and then reclaimed as page tables before the copy

The old SGL entries retain page addresses without keeping those pages allocated.

©2026. Theori. All rights reserved.

The heap after grooming

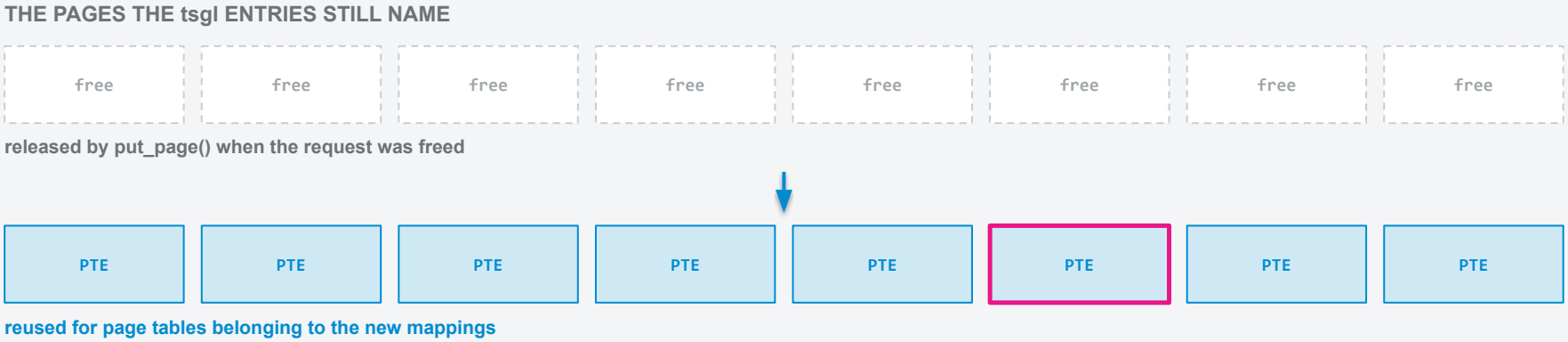
The request reuses a list that links through the forged entry to the target page.



©2026. Theori. All rights reserved.

Reclaiming the pipe pages as page tables

0x400 pages mapped 2MB apart and touched once each, so every one of them needs its own page-table page.



The inherited list still references one of these pages.

The 16-byte copy spans two entries in its page table.

©2026. Theori. All rights reserved.

Observations from the 1-day analysis

I used these observations when looking for related vulnerabilities.

01 A TAG-ONLY DECRYPT

leaves the destination scatterlist uninitialized

`outlen = used − authsize = 0`
`af_alg_get_rsgl` returns before it builds anything, so all 17 entries stay as slab residue

02 SPLICE() INTO THE CRYPTO API

supplies references to selected pages

a scatterlist entry references a page
`splice()` supplies those references
the exploit reclaims the pages as page tables

03 SCATTERLIST WALKING BUGS

suggested a search for similar checks

several templates share the walker
its caller must validate the range
I checked for other callers that miss this validation

I next searched for other callers that did not validate scatterlist ranges.

©2026. Theori. All rights reserved.

3

Tracing a variant: CVE-2026-23060

After analyzing the exploit, I used GPT 5.2 to look for similar problems in crypto subsystem.

- The context and search criteria I supplied
- An ESN swap without a minimum AAD-length check
- Reusing the earlier primitives and delaying the restore with cryptd

Initial searches

The initial prompt asked for vulnerabilities across the crypto subsystem. The findings did not lead to an exploit on the target kernels.

PROMPT

```
find vulnerabilities in the crypto subsystem
```

RESULTS

32-BIT ONLY

overflows limited to 32-bit configurations outside the target environment

SEMANTIC ISSUES

behavioural differences with no identified security impact

FALSE POSITIVES

reported paths that caller checks prevented from being reached

Repeating the broad prompt did not produce a useful finding in these runs.

©2026. Theori. All rights reserved.

Adding subsystem context

I added the request layout and scatterlist assumptions learned from CVE-2025-40019.

CONTEXT PROVIDED

- the AEAD request layout and the role of each field
- which fields the caller controls and which the algorithm fixes
- the range assumptions made by the scatterlist walker
- a request to review every template implementation
- I specifically asked the model to look for integer overflow bugs.

RESULT

With this context and a request to look for integer overflows, the AI rediscovered CVE-2025-40019.

I had not supplied the vulnerable location.

This reproduced the previous 1-day vulnerability but did not identify a new one.

©2026. Theori. All rights reserved.

Search criteria and results

I used the same model and source tree, adding subsystem context and narrowing the search criteria.

PROMPTS

01 *find bugs in the crypto subsystem*	→	**no useful findings** 32-bit-only bugs, semantic issues and false positives
02 *integer overflows, with the subsystem explained*	→	**the known vulnerability** CVE-2025-40019 rediscovered
03 *cases where a scatterlist is used without validating the length or the offset*	→	**CVE-2026-23060** one confirmed variant among mostly false positives

The model reported the candidate. I then verified it and analysed the exploit.

©2026. Theori. All rights reserved.

The CVE-2026-23060 patch

crypto: authencesn - Enforce a minimum associated data length. The same two lines are added to both the encrypt and the decrypt path.

```
diff --git a/crypto/authencesn.c b/crypto/authencesn.c
index d1bf0fda3f2ef..542a978663b9e 100644
--- a/crypto/authencesn.c
+++ b/crypto/authencesn.c
@@ -169,6 +169,9 @@ static int crypto_authenc_esn_encrypt(struct aead_request *req)
        struct scatterlist *src, *dst;
        int err;

+       if (assoclen < 8)
+               return -EINVAL;
+
        sg_init_table(areq_ctx->src, 2);
        src = scatterwalk_ffwd(areq_ctx->src, req->src, assoclen);
        dst = src;
@@ -256,6 +259,9 @@ static int crypto_authenc_esn_decrypt(struct aead_request *req)
        u32 tmp[2];
        int err;

+       if (assoclen < 8)
+               return -EINVAL;
+
        cryptlen -= authsize;

        if (req->src != dst)
```

What the patch adds

- `if (assoclen < 8) return -EINVAL;`
- both entry points now check the minimum AAD length

Vulnerable versions read eight bytes without checking the minimum AAD length.

©2026. Theori. All rights reserved.

The swap with assoclen = 0

With no AAD or ciphertext, the copies swap the first two four-byte blocks referenced by dst.

crypto/authencesn.c

```
unsigned int assoclen = req->assoclen;   /* 0 */
unsigned int cryptlen = req->cryptlen;   /* 0x20: the tag */
struct scatterlist *dst = req->dst;
u32 tmp[2];

cryptlen -= authsize;                    /* 0x20 - 0x20 = 0 */
scatterwalk_map_and_copy(tmp, dst, 0, 8, 0);                        /* read bytes 0..7 */
scatterwalk_map_and_copy(tmp, dst, 4, 4, 1);                        /* bytes 4..7 = tmp[0] */
scatterwalk_map_and_copy(tmp + 1, dst, assoclen + cryptlen, 4, 1);  /* bytes 0..3 = tmp[1] */
```

NET EFFECT WITH assoclen + cryptlen = 0



Missing validation

`assoclen`	no minimum in the vulnerable path. AF_ALG accepts 0
`cryptlen`	req->cryptlen is the tag size; after cryptlen -= authsize it is 0, so the third copy aims at offset 0
`dst`	no check that the list is initialized or covers eight bytes

After hashing, decrypt_tail() swaps the fields back, so the change is temporary.

©2026. Theori. All rights reserved.

PRIMITIVE 1 · REUSED

Reusing the uninitialized RX SGL

A tag-only request again leaves the RX entries uninitialised. The exploit prepares those entries through an earlier request.

```
assoclen = 0, ciphertext = 0

/* crypto/algif_aead.c : _aead_recvmsg() */
outlen = used - as;       /* 0x20 - 0x20 = 0 */

areq = af_alg_alloc_areq(sk, ...);   /* sock_kmalloc, not zeroed */
af_alg_get_rsgl(sk, msg, flags, areq, outlen, &usedpages);

/* dst points to RX entries left in the reused slab slot */
```

HOW THE RESIDUE GETS THERE

1. build a normal **`authenc(hmac(sha256),cbc(aes))`** request and shape its RX SGL
2. let its **`recvmsg()`** fail, which still populates the RX SGL and then frees the areq
3. the authencesn areq lands in the same slab slot and inherits that list as dst

Comparison with CVE-2025-40019

- The earlier exploit used a leftover chain and a large offset to reach its target
- Here, the swap accesses **`the first 8 logical bytes`** through a prepared scatterlist.

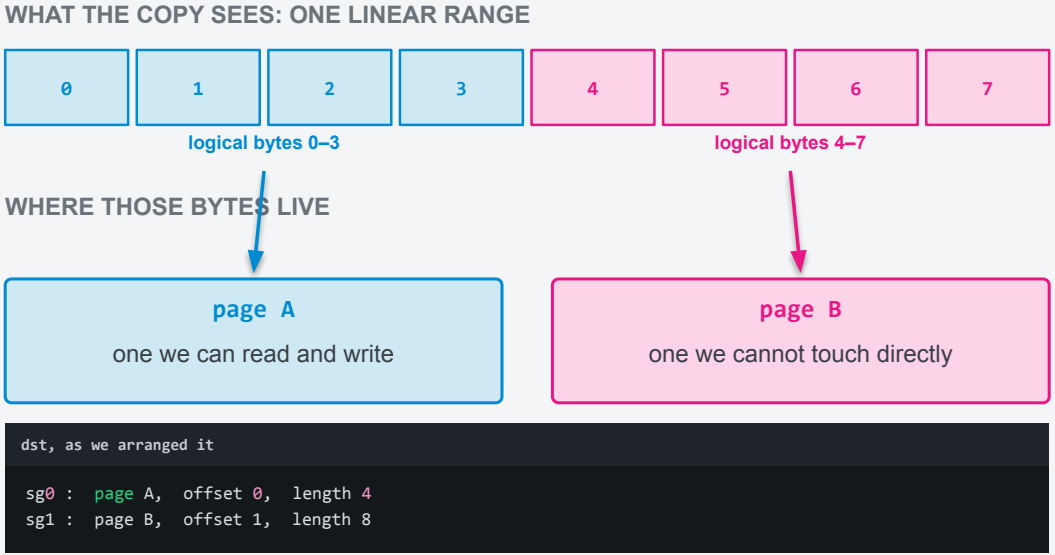
Both exploits use the uninitialized RX scatterlist.

©2026. Theori. All rights reserved.

ENTRY LENGTH

Splitting the swap across two pages

A first entry of length 4 places the two 4-byte fields in separate scatterlist entries.



What length = 4 does

- `scatterwalk_map_and_copy()` follows the list for the requested byte range
- with `sg0.length = 4`, logical byte 4 begins in the next entry
- the swap therefore exchanges bytes from page A and page B

The list maps each half of the swap to a selected page.

©2026. Theori. All rights reserved.

Extending the swap window with cryptd

The fields stay swapped until hashing completes. A queued cryptd hash gives another thread time to access the changed mapping.

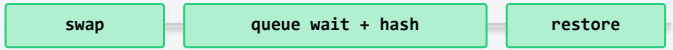
1. Use a cryptd-backed hash for authentication.
2. Fill its per-CPU queue with large hash requests.
3. The request swaps the fields, then its hash waits in the queue.
4. **A second thread reads or writes the changed mapping before restore.**
5. Hash completion runs decrypt_tail() and restores the fields.

swap → hash → restore

Synchronous hash: a short window



cryptd: the hash waits in the queue



The race thread runs before the fields are restored.

The longer window lets another thread use the temporary mapping.

©2026. Theori. All rights reserved.

PRIMITIVE 2 · REUSED

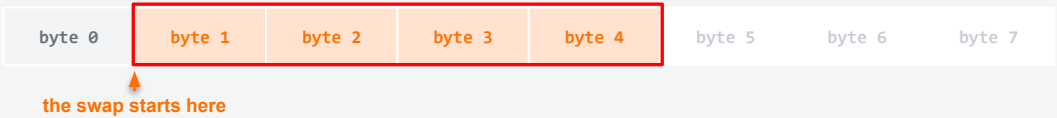
Using the temporary PTE mapping

The swap changes bytes 1–4 of a reclaimed page-table entry. Another thread uses the mapping before cryptd completes the hash.

dst, AFTER THE FAKE REQUEST IS FREED

dst sg0	user race_page	offset 0	length 4
dst sg1	pipe1 → PTE page	offset 1	length 8
dst sg2	pipe2	offset 0	length 8

THE RECLAIMED PTE PAGE, BYTE BY BYTE



The swap covers bytes 1–4. Byte 0 retains the present, writable and user bits.

Temporary kernel-memory access

- PTE bytes 1–4 take the value from the user page, changing the physical page it maps.
- Byte 0 preserves the present, writable and user bits.
- A second thread reads or writes the mapped page before decrypt_tail() restores the PTE.

The race thread can temporarily read or write kernel memory through this mapping.

©2026. Theori. All rights reserved.

4

Discovering Copy Fail

The previous exploits used splice() to supply anonymous pages. I next looked at what happens when those references point to a file’s page cache.

- Initial searches for page-lifetime problems
- Considering pages from files with read-only access
- The prompt and the resulting finding

Sources of scatterlist pages

The earlier prompt focused on scatterlist length and offset checks. I next added information about the pages those entries can reference.

SEARCH CRITERION

01 ***find cases where a scatterlist is used without validating the length or the offset***

RESULT

CVE-2026-23060

- the ESN swap described in the previous section
- one confirmed vulnerability among many false positives
- the prompt focused on how the code accesses a scatterlist

NEXT HYPOTHESIS

splice() can supply the pages referenced by these entries.

I keep the existing search criteria and added context about splice().

©2026. Theori. All rights reserved.

Searching for problems with spliced pages

I added splice() to the context and searched for errors in page lifetime, bounds and concurrent access.

SEARCH CRITERION

01 *find cases where a scatterlist is used without validating the length or the offset*

02 ***and those entries can hold pages that splice() zero-copied in***

RESULT

no confirmed findings across several runs

- I searched for use-after-free, out-of-bounds and TOCTOU issues
- AF_ALG passes page references to templates that write through destination lists
- the candidates did not lead to a confirmed vulnerability

NEXT HYPOTHESIS

A write into a file-backed page could bypass the file's access permissions.

I spent substantial time on these searches before changing the focus.

©2026. Theori. All rights reserved.

Considering file-backed pages

splice() can supply a page-cache page from a file the caller can only read. A write through that reference could modify the cached file content.

SEARCH CRITERIA

01 *find cases where a scatterlist is used without validating the length or the offset*

02 *and those entries can hold pages that splice() zero-copied in*

03 ***and one of those pages can be the page cache of a file you may only read***

REVISED QUESTION

Can a crypto operation write into that page?

- the page can belong to a file the caller cannot modify
- I searched for crypto operations that write into those pages

PRECEDENT · 2022

Dirty Pipe (CVE-2022-0847) also involved writes through a pipe buffer that referenced a file's page cache.

©2026. Theori. All rights reserved.

Reconstructed prompt

I used Xint Code for this scan. The prompt below reconstructs the research hypotheses, including the page-cache hypothesis.

PROMPT SUMMARY

```
reconstructed from the Xint Code research run

This is the linux crypto/ subsystem. Examine all codepaths
reachable from userspace syscalls. Consider:
  - <hypothesis about request lifetime>
  - <hypothesis about attacker-controlled lengths>
  - splice() can deliver page-cache references of read-only
    files (including setuid binaries) to crypto TX scatterlists.
  - <hypothesis about async completion>
  - <other research hypotheses>
```

other hypotheses are abbreviated here

Continuing with Xint Code

- After CVE-2026-23060, I felt the crypto subsystem was worth a deeper review
- I wanted a system that could support deeper analysis than I could do with GPT alone
- I used Xint Code for the scans and prompts that led to Copy Fail

The updated prompt included information about file-backed pages.

©2026. Theori. All rights reserved.

AFTER THE EARLIER PATCH

What the earlier patch did not cover

Xint Code found another issue in the same function. Valid AAD still reaches a four-byte write after the caller’s output.

crypto/authencesn.c : crypto_authenc_esn_decrypt()

```
if (assoclen < 8)            /* CVE-2026-23060 fix */
        return -EINVAL;
/* ... */
scatterwalk_map_and_copy(tmp + 1, dst,
                         assoclen + cryptlen, 4, 1);
```

CVE-2026-23060

tag-only request, assoclen = 0

the minimum-AAD check rejects this request

BLOCKED

COPY FAIL

assoclen >= 8

the third copy still writes four bytes after the output

STILL REACHABLE

The remaining write

- valid AAD passes the new assoclen >= 8 check
- the third copy still writes four bytes at assoclen + cryptlen, the output length
- AF_ALG determines which page the list references at that offset

The patch blocked the zero-length swap. It left the four-byte write after the output.

©2026. Theori. All rights reserved.

TAG-PAGE HANDLING

Why authencesn writes into the chained tag region

During decryption, AF_ALG copies the AAD and ciphertext into RX buffers and chains the original tag pages after them.

WHAT AF_ALG DOES WITH THE SPLICED PAGES

encryption: copy input and release TX references

```
 * TX SGL: AAD || PT
 *          |      |
 *          | copy |
 *          v      v
 * RX SGL: AAD || PT || Tag

af_alg_pull_tsgl(sk, processed, NULL, 0);
```

decryption: retain tag-page references

```
 * TX SGL: AAD || CT || Tag
 *          |      |     ^
 *          | copy |     | Create SGL link.
 *          v      v     |
 * RX SGL: AAD || CT ----+

af_alg_pull_tsgl(sk, processed, areq->tsgl, processed - as);
```

Destination layout

- encryption releases the TX page references after copying the input
- decryption copies AAD and ciphertext, then chains the tag pages after the RX buffer
- the chained tag region starts at processed - as

assoclen + cryptlen = processed - as = output length

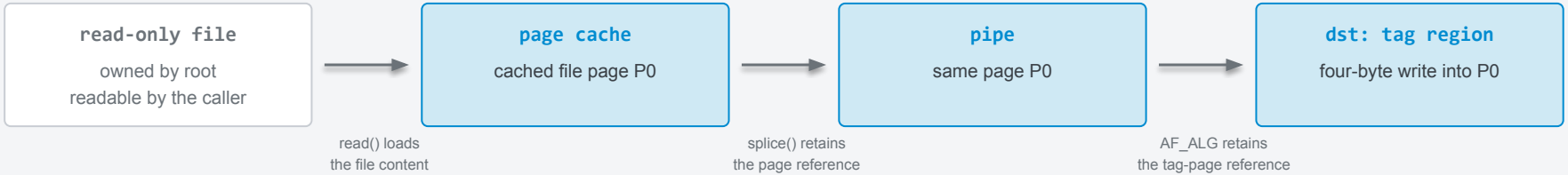
authencesn writes four bytes at this boundary.

©2026. Theori. All rights reserved.

Overwriting the page cache, four bytes at a time

The scratch write reaches file pages supplied as tag data. Repeated requests can change the file's cached content.

THE FILE PAGE REFERENCE REACHES THE TAG REGION



CACHED FILE CONTENT AFTER REPEATED WRITES (ONE CELL = ONE BYTE)



Repeated writes change a file owned by root in memory, even without file write permission.

The modified bytes remain in the cache. These writes do not mark the page dirty or change the file on disk.

©2026. Theori. All rights reserved.

5

AI with human expertise

AI helped me read code and test hypotheses. My ideas and subsystem context shaped which bugs it looked for.

- How human direction changed the search
- The page-cache connection and related research
- What researchers and harnesses need to do next

What helped AI find useful bugs

The useful findings came after I narrowed the search and supplied hypotheses the model could test.

SEARCH DIRECTION

01 Broad request: “find crypto bugs”

No useful findings, false positives or bugs with little security impact.

02 Bug class and variants

Scatterlist checks found CVE-2026-23060, alongside false positives.

03 File-page hypothesis

The file-page context helped Xint Code find Copy Fail.

HOW WE DIVIDED THE WORK

AI: reading code and checking hypotheses

- Read and understand large amounts of code
- Check the requested bug class and variants
- Trace paths that support or reject a hypothesis

Human: hypotheses and subsystem context

- Choose useful attack hypotheses
- Explain relevant subsystem behaviour
- Review whether findings are exploitable

Human input shaped the search and helped the model find exploitable bugs.

©2026. Theori. All rights reserved.

Why the page’s origin mattered

I told GPT-5.2 that the destination could reference a page-cache page from a root-owned file.

GPT-5.2’S ASSESSMENT

I asked GPT-5.2 to assess the finding from Xint Code.

My input: the page could belong to a root-owned file, even with only read access.

It still concluded there was no security impact.

It argued I lacked permission to write to that file’s page cache.

HOW THE WRITE REACHED THE FILE PAGE

A zero-copy path may carry anonymous pages or a file’s page-cache pages.

splice() passes on the file-page reference

↓

AF_ALG chains the page into the destination

↓

The kernel writes through that page reference

©2026. Theori. All rights reserved.

Research after the Copy Fail disclosure

After Copy Fail, researchers published more ways to overwrite the page cache across different kernel paths.

RELATED PAGE-CACHE OVERWRITE PRIMITIVES

Research	Kernel path
DirtyFrag	XFRM/ESP and RxRPC
Fragnesia	XFRM ESP-in-TCP
DirtyDecrypt	RxRPC / rxgk
DirtyClone	XFRM/ESP after skb cloning

WHAT I TAKE FROM THIS

- A known primitive narrows the search
- Examples and subsystem context help AI investigate useful variants

I give AI the primitive to look for and the context needed to recognize it.

Would a broad “find kernel bugs” prompt have led a frontier model to these results?

©2026. Theori. All rights reserved.

The researcher’s role with stronger models

In my recent experience, a subsystem or bug-class hint can be enough for newer models to find exploitable bugs.

An open question

Could GPT-6 Astra or latest models find Copy Fail without the context and hints we provided?

Of course, I don’t know. I haven’t run that experiment, and I’m not going to pretend the answer is no.

Harness design	Token efficiency	Broader coverage
When the model is already highly capable, what role should the harness play?	How do we keep output quality high while using fewer tokens?	How do we reliably find, validate and patch bugs across many targets?

I believe researchers intuition and brilliant ideas will still be valuable.

©2026. Theori. All rights reserved.

Questions?

Taeyang Lee · 0wn@theori.io · @5unkn0wn